

grid_coordinate_simulation_printable_answers

Grid 題型與座標模擬

Status: in_use

Prereq: list、for、if、函式、基本輸入輸出

這份講義保留 grid 題最必要的語法、觀念與 APCS 完整解法，特別對應中級題本：

- 〈特殊位置〉：2D grid、曼哈頓距離、範圍加總
- 〈蒐集寶石〉：座標模擬、方向狀態、右轉、牆壁與邊界

1. Grid 是什麼？

Grid 就是二維表格。APCS 題目常把地圖、矩陣、場地、棋盤寫成 grid。

在 Python 中，grid 通常用 list of lists 表示：

```
grid = [  
    list("S..#."),  
    list(".#..G"),  
    list("...#."),  
    list("G...."),  
]  
  
for row in grid:  
    print("".join(row))
```

```
S..#.  
.#..G  
...#.  
G....
```

2. Row / Column / 座標

Grid 題常用 (r, c) 表示座標：

- r：row，列，由上到下
- c：column，欄，由左到右
- Python 索引從 0 開始

取值方式是 `grid[r][c]`。

```
print(grid[0][0]) # S
print(grid[1][4]) # G
print("rows =", len(grid))
print("cols =", len(grid[0]))
```

```
S
G
rows = 4
cols = 5
```

3. 走訪所有格子

走訪 grid 最常用雙層迴圈：外層走列 r，內層走欄 c。

```
for r in range(len(grid)):
    for c in range(len(grid[0])):
        print(r, c, grid[r][c])
```

```
0 0 S
0 1 .
0 2 .
0 3 #
0 4 .
1 0 .
1 1 #
1 2 .
1 3 .
1 4 G
2 0 .
2 1 .
2 2 .
2 3 #
2 4 .
3 0 G
3 1 .
3 2 .
3 3 .
3 4 .
```

4. 常用語法整理

list("S..#.")

把字串拆成字元 list。

"".join(row)

把字元 list 合回字串，適合印地圖。

tuple unpacking

`dr, dc = (0, 1)` 會把 tuple 裡兩個值分別放進 `dr` 和 `dc`。

`.append(...)`

把答案、座標或路徑加到 list 後面。

`[row[:] for row in grid]`

複製整張 grid，避免模擬時改到原本資料。

```
row = list("S..#.")
print(row)
print("".join(row))

dr, dc = (0, 1)
print(dr, dc)

path = []
path.append((0, 0))
path.append((0, 1))
print(path)

grid_copy = [row[:] for row in grid]
grid_copy[0][0] = "."
print("原本:", grid[0][0])
print("複製:", grid_copy[0][0])
```

```
['S', '.', '.', '#', '.']
S..#.
0 1
[(0, 0), (0, 1)]
原本: S
複製: .
```

5. 邊界檢查

只要座標可能移動，就一定要先檢查有沒有出界。

正確順序：先檢查 `in_bounds`，再取 `grid[r][c]`。

```
def in_bounds(grid, r, c):
    return 0 <= r < len(grid) and 0 <= c < len(grid[0])

print(in_bounds(grid, 0, 0))
print(in_bounds(grid, -1, 0))
print(in_bounds(grid, 0, 99))
```

```
True
False
False
```

6. 上下左右移動

如果指令是 U, D, L, R，可以用 dictionary 建方向表。

```
DIRECTIONS_COMMAND = {
    "U": (-1, 0),
    "D": (1, 0),
    "L": (0, -1),
    "R": (0, 1),
}

def next_position_by_command(r, c, command):
    dr, dc = DIRECTIONS_COMMAND[command]
    return r + dr, c + dc

print(next_position_by_command(2, 3, "U"))
print(next_position_by_command(2, 3, "R"))
```

```
(1, 3)
(2, 4)
```

7. 曼哈頓距離

兩個格子 (r, c) 和 (s, t) 的曼哈頓距離：

```
abs(r - s) + abs(c - t)
```

常見錯誤是寫成 `abs(r - s + c - t)`，這會讓正負差互相抵消。

```
def distance(r, c, s, t):
    return abs(r - s) + abs(c - t)

print(distance(2, 3, 0, 1))
print(distance(0, 0, 0, 2))
```

```
4
2
```

8. APCS 〈特殊位置〉題型拆解

這題不是單純「會用 grid」就好，重點是把文字條件翻成枚舉範圍。

對每個位置 (i, j)：

1. 令 $x = \text{mat}[i][j]$ 。
2. 找出所有與 (i, j) 曼哈頓距離不超過 x 的位置 (s, t) 。
3. 把這些位置的數值加總成 total 。
4. 如果 $\text{total} \% 10 == x \% 10$ ，或等價寫成 $(\text{total} - x) \% 10 == 0$ ，則 (i, j) 是特殊位置。

題目要求輸出 row-major 順序，也就是由上到下、由左到右。只要外層 `for i in range(n)`、內層 `for j in range(m)`，找到時依序 `append`，輸出順序自然正確。

8-1. 第一子題： $n = 1$ 的一維情境

第一子題有 60 分，條件是 $n = 1$ ，也就是只有一列。

例如：

```
+---+---+---+---+---+
| 3 | 1 | 4 | 5 | 1 |
+---+---+---+---+---+
  0  1  2  3  4
```

因為只有一列，所以每個位置只需要看左右範圍：

```
left = max(0, j - x)
right = min(m - 1, j + x)
```

這一版可以先掌握第一子題，也能看出「距離不超過 x 」其實就是範圍限制。

```
def solve_special_one_row(values):
    m = len(values)
    points = []

    for j in range(m):
        x = values[j]
        left = max(0, j - x)
        right = min(m - 1, j + x)
        total = sum(values[left:right + 1])

        if (total - x) % 10 == 0:
            points.append(j)

    output = [str(len(points))]
    for j in points:
        output.append(f"0 {j}")
    return "\n".join(output)

print(solve_special_one_row([3, 1, 4, 5, 1]))
```

```
2
0 0
0 2
```

8-2. 從一維推到二維：為什麼要枚舉方框？

對中心 (i, j) 、半徑 x 而言，如果某點 (s, t) 的距離不超過 x ，必定滿足：

```
i - x <= s <= i + x
j - x <= t <= j + x
```

所以可以先枚舉這個方框：

```
for s in range(i - x, i + x + 1):
    for t in range(j - x, j + x + 1):
        ...
```

但注意：在方框內不代表一定距離不超過 x 。方框角落可能太遠，所以還要補兩個檢查：

1. $0 \leq s < n$ and $0 \leq t < m$ ：沒有出界。
2. $\text{abs}(i - s) + \text{abs}(j - t) \leq x$ ：真的在曼哈頓距離內。

8-3. 官方範例二：檢查 (2, 3)

範例二矩陣：

```
1 3 4 1 3 1
1 1 4 1 3 1
1 1 3 2 5 3
4 3 3 1 4 1
5 2 1 1 1 1
```

題目說 $(2, 3)$ 是特殊位置。因為 $\text{mat}[2][3] = 2$ ，所以要加總距離不超過 2 的所有格子。

下面列出實際被加總的座標、值與距離。

```

special_sample2_mat = [
    [1, 3, 4, 1, 3, 1],
    [1, 1, 4, 1, 3, 1],
    [1, 1, 3, 2, 5, 3],
    [4, 3, 3, 1, 4, 1],
    [5, 2, 1, 1, 1, 1],
]

center_r, center_c = 2, 3
x = special_sample2_mat[center_r][center_c]
total = 0
used = []

for s in range(center_r - x, center_r + x + 1):
    for t in range(center_c - x, center_c + x + 1):
        if 0 <= s < len(special_sample2_mat) and 0 <= t < len(special_sample2_mat[0]):
            dist = abs(center_r - s) + abs(center_c - t)
            if dist <= x:
                value = special_sample2_mat[s][t]
                used.append((s, t, value, dist))
                total += value

print("x =", x)
print("被加總的格子數 =", len(used))
for item in used:
    print(item)
print("total =", total)
print("total % 10 =", total % 10)
print("x % 10 =", x % 10)

```

```

x = 2
被加總的格子數 = 13
(0, 3, 1, 2)
(1, 2, 4, 2)
(1, 3, 1, 1)
(1, 4, 3, 2)
(2, 1, 1, 2)
(2, 2, 3, 1)
(2, 3, 2, 0)
(2, 4, 5, 1)
(2, 5, 3, 2)
(3, 2, 3, 2)
(3, 3, 1, 1)
(3, 4, 4, 2)
(4, 3, 1, 2)
total = 32
total % 10 = 2
x % 10 = 2

```

8-4. 完整解法：方框枚舉版

這是最適合本題的 Python 寫法：夠直觀，也比掃整張 grid 更有效率。

重點：

- `area_sum_box(mat, r, c)` 負責算某個中心的距離內總和。
- `find_special_positions(mat)` 負責走訪全部中心點並收集答案。
- `solve_special_position(lines)` 負責處理輸入輸出格式。

```
def area_sum_box(mat, r, c):
    n = len(mat)
    m = len(mat[0])
    x = mat[r][c]
    total = 0

    for s in range(r - x, r + x + 1):
        for t in range(c - x, c + x + 1):
            if 0 <= s < n and 0 <= t < m:
                if abs(r - s) + abs(c - t) <= x:
                    total += mat[s][t]

    return total

def find_special_positions(mat):
    points = []
    for r in range(len(mat)):
        for c in range(len(mat[0])):
            x = mat[r][c]
            total = area_sum_box(mat, r, c)
            if (total - x) % 10 == 0:
                points.append((r, c))
    return points

def solve_special_position(lines):
    n, m = map(int, lines[0].split())
    mat = [list(map(int, lines[i].split())) for i in range(1, n + 1)]

    points = find_special_positions(mat)
    output = [str(len(points))]
    for r, c in points:
        output.append(f"{r} {c}")
    return "\n".join(output)
```

8-5. 官方範例驗證

範例一是第一子題型態 $n = 1$ ，範例二是完整二維情境。

```

special_sample1 = """
1 5
3 1 4 5 1
""".strip().splitlines()

special_sample2 = """
5 6
1 3 4 1 3 1
1 1 4 1 3 1
1 1 3 2 5 3
4 3 3 1 4 1
5 2 1 1 1 1
""".strip().splitlines()

print(solve_special_position(special_sample1))
print("----")
print(solve_special_position(special_sample2))

```

```

2
0 0
0 2
---
4
0 2
1 3
2 3
3 3

```

8-6. 整合輸入輸出的程式碼

前面用 `solve_special_position(lines)` 方便測試；正式讀取資料時，可改成直接讀 `input()` 的寫法。

```

# 整合輸入輸出的寫法

n, m = map(int, input().split())
mat = []
for _ in range(n):
    mat.append([int(x) for x in input().split()])

points = []
for r in range(n):
    for c in range(m):
        x = mat[r][c]
        total = 0
        for s in range(r - x, r + x + 1):
            for t in range(c - x, c + x + 1):
                if 0 <= s < n and 0 <= t < m and abs(r - s) + abs(c - t) <= x:
                    total += mat[s][t]
        if (total - x) % 10 == 0:
            points.append((r, c))

print(len(points))
for r, c in points:
    print(r, c)

```

9. 方向狀態與右轉

如果題目需要轉向，用整數表示方向很方便：

- 0：東
- 1：南
- 2：西
- 3：北

右轉就是 $(\text{direction} + 1) \% 4$ 。

```
DIRECTIONS_TURN = [
    (0, 1),
    (1, 0),
    (0, -1),
    (-1, 0),
]

def turn_right(direction):
    return (direction + 1) % 4

for d in range(4):
    print(d, "右轉後", turn_right(d))
```

```
0 右轉後 1
1 右轉後 2
2 右轉後 3
3 右轉後 0
```

10. APCS 〈蒐集寶石〉完整解法

這題是狀態模擬。狀態包含：

- 位置 (r, c)
- 方向 direction
- 得分 score
- 已取得寶石數 gem_count
- 每格剩餘寶石數 $\text{grid}[r][c]$

注意：0 不是牆壁，可以走進去；只是走到後下一回合停止。牆壁是 -1。

```

def next_position_by_direction(r, c, direction):
    dr, dc = DIRECTIONS_TURN[direction]
    return r + dr, c + dc

def can_move_forward(mat, r, c, direction):
    nr, nc = next_position_by_direction(r, c, direction)
    return in_bounds(mat, nr, nc) and mat[nr][nc] != -1

def rotate_until_can_move(mat, r, c, direction):
    while not can_move_forward(mat, r, c, direction):
        direction = turn_right(direction)
    return direction

def simulate_gems(mat, K, start_r, start_c):
    r, c = start_r, start_c
    direction = 0
    score = 0
    gem_count = 0

    while True:
        if mat[r][c] == 0:
            break

        score += mat[r][c]
        mat[r][c] -= 1
        gem_count += 1

        if score % K == 0:
            direction = turn_right(direction)

        direction = rotate_until_can_move(mat, r, c, direction)
        r, c = next_position_by_direction(r, c, direction)

    return gem_count

def solve_gems(lines):
    M, N, K, r, c = map(int, lines[0].split())
    mat = [list(map(int, lines[i].split())) for i in range(1, M + 1)]
    return str(simulate_gems(mat, K, r, c))

```

```

gems_sample1 = """
1 7 3 0 4
1 -1 2 1 2 1 0
""".strip().splitlines()

gems_sample2 = """
4 5 4 2 1
2 0 1 1 1
2 -1 0 2 -1
0 3 2 3 0
-1 1 -1 3 0
""".strip().splitlines()

print(solve_gems(gems_sample1))
print(solve_gems(gems_sample2))

```

11. Fix-it 題與解答

這一段整理 grid 題最常見的錯誤。每題先看錯誤寫法，再看修正版。

Fix-it 1：欄數寫成列數

錯誤原因：len(grid) 是列數，不是欄數。當 grid 不是正方形時，內層迴圈會漏算或超出範圍。

錯誤寫法

```
def count_all_cells_bug(grid):  
    count = 0  
    for r in range(len(grid)):  
        for c in range(len(grid)): # 錯：這裡應該是欄數  
            count += 1  
    return count
```

修正版

```
def count_all_cells_fixed(grid):  
    count = 0  
    for r in range(len(grid)):  
        for c in range(len(grid[0])):  
            count += 1  
    return count  
  
rect_grid = [  
    [1, 2, 3],  
    [4, 5, 6],  
]  
  
print(count_all_cells_fixed(rect_grid)) # 2 * 3 = 6
```

Fix-it 2：還沒檢查邊界就讀取 grid

錯誤原因：如果 nr, nc 已經出界，先讀 grid[nr][nc] 會直接發生錯誤。正確順序是先檢查範圍，再讀格子內容。

錯誤寫法

```
def can_move_bug(grid, r, c, dr, dc):  
    nr = r + dr  
    nc = c + dc  
    if grid[nr][nc] == "#": # 錯：可能已經出界  
        return False  
    if not in_bounds(grid, nr, nc):  
        return False  
    return True
```

修正版

```
def can_move_fixed(grid, r, c, dr, dc):  
    nr = r + dr  
    nc = c + dc  
    if not in_bounds(grid, nr, nc):  
        return False  
    if grid[nr][nc] == "#":  
        return False  
    return True  
  
print(can_move_fixed(grid, 0, 0, -1, 0)) # 往上出界  
print(can_move_fixed(grid, 0, 0, 0, 1)) # 往右可走
```

Fix-it 3：曼哈頓距離寫錯

錯誤原因： $\text{abs}(r - s + c - t)$ 會讓列差與欄差互相抵消。曼哈頓距離必須分開取絕對值再相加。

錯誤寫法

```
def distance_bug(r, c, s, t):  
    return abs(r - s + c - t)
```

修正版

```
def distance_fixed(r, c, s, t):  
    return abs(r - s) + abs(c - t)  
  
print(distance_bug(0, 3, 2, 1)) # 錯誤結果：0  
print(distance_fixed(0, 3, 2, 1)) # 正確結果：4
```

Fix-it 4：把 0 當成牆壁

在〈蒐集寶石〉中，-1 才是牆壁。0 是可以走進去的格子，只是走到後會停止。

錯誤寫法

```
def can_move_forward_bug(mat, r, c, direction):
    nr, nc = next_position_by_direction(r, c, direction)
    return in_bounds(mat, nr, nc) and mat[nr][nc] > 0 # 錯:0 也可以走
```

修正版

```
def can_move_forward_fixed(mat, r, c, direction):
    nr, nc = next_position_by_direction(r, c, direction)
    return in_bounds(mat, nr, nc) and mat[nr][nc] != -1
```

```
one_row = [[1, -1, 2, 1, 2, 1, 0]]
print(can_move_forward_fixed(one_row, 0, 5, 0)) # 往右到 0，可以走
```

Fix-it 5：收寶石與移動順序顛倒

在〈蒐集寶石〉中，每一回合要先處理目前格子，再決定方向與移動。不能先移動到下一格才收寶石。

錯誤觀念：先移動，再處理新格子

```
# r, c = next_position_by_direction(r, c, direction)
# score += mat[r][c]
```

正確順序：先處理目前格子

```
def one_step_collect(mat, r, c, score, gem_count):
    score += mat[r][c]
    mat[r][c] -= 1
    gem_count += 1
    return score, gem_count
```

```
demo = [[2]]
score, gem_count = one_step_collect(demo, 0, 0, 0, 0)
print(score)
print(gem_count)
print(demo[0][0])
```

12. Grid 題解題 SOP

1. 確認 grid 大小：rows, cols
2. 確認座標定義：(r, c) 是列、欄
3. 確認格子值的意義：牆壁、空地、數字、目標
4. 寫 in_bounds
5. 寫方向表與下一格函式
6. 寫 can_move
7. 寫單一步驟
8. 寫整體迴圈或雙層掃描
9. 用官方範例驗證
10. 最後整理輸入輸出